Single-precision formal solvers

October 2026

The linear, quadratic and Hermite (cubic Bézier) long-characteristic formal solvers in SRC/rt_solvers_mod.f90 evaluate their interval weights in single precision (fp32) without data-dependent branches. Every weight is accurate to a few units in the last place (ulp) over the whole range of optical depths that occurs, including the bounded maser intervals with negative optical depth. This note gives the motivation, the formulation and its accuracy.

1 MOTIVATION

Across one interval of a characteristic, the formal solution is

$$I_c = I_u\, e^{-\Delta} + \int_0^{\Delta} S(t)\, e^{-t}\, \mathrm{d}t, \tag{1.1}$$

where Δ is the optical depth of the interval, t is measured backward from the target point c, and u denotes the upwind point. Each scheme replaces $S(t)$ by an interpolant in a few source values and slopes, so that I_c is a weighted sum of these with weights that depend only on Δ. The weights are evaluated once per frequency and grid point, which makes them the innermost work of every solver.

The opacities, source functions and intensities are stored in fp32. The weights were nevertheless formed in fp64, because their textbook expressions cancel: $1 - e^{-\Delta}$ loses all relative accuracy as $\Delta \to 0$, and the moment combinations behind the higher-order weights cancel to $O(\Delta^n)$ with n up to 4. The fp64 code therefore also switched to a truncated Taylor series below a threshold and clamped the exponential at $\Delta = 80$. Both the branches and the fp64 arithmetic halve or remove the SIMD width of the frequency loop, and in fp64 the arithmetic cost is no longer negligible against the memory traffic of the fp32 arrays. The previous linear step, for example, read

```
dt = dsup05*(xxup + xx)
dt = max(dt, -maser_dtau_max)
if (dt < 80) then
  ex = exp(-dt)
  if (abs(dt) > 1d-3) then
    u0 = 1 - ex
    Psi = 1 - u0/dt
  else
```

```
    Psi = dt*(0.5 - 0.16666667*dt)
    u0 = dt*(1 - Psi)
  endif
  Inu = Inuup*ex + (u0 - Psi)*ssup + Psi*ss
else
  u0 = 1/dt
  Psi = 1 - u0
  Inu = u0*ssup + Psi*ss
endif
```

The approach below removes every cancellation analytically, so that fp32 suffices. All forms are written so that both sides of a switch are evaluated and one is selected without a branch.

2 The shared exponential

All three schemes need $E = e^{-\Delta}$ and $U_0 = 1 - e^{-\Delta}$, each to full relative accuracy, for Δ between the maser cap (-2 by default) and the optically thick limit. They are computed together by range reduction (`lw_exp`). With

$$\Delta = n \ln 2 + r, \qquad n = \mathrm{nint}(\Delta / \ln 2), \qquad |r| \le \tfrac{1}{2} \ln 2, \tag{2.1}$$

where $\ln 2$ is split into an exactly representable leading part and a small correction (Cody–Waite), so that r is exact to rounding, and

$$s = 2^{-n}, \qquad m = 1 - e^{-r} = r\, q(r), \tag{2.2}$$

one has

$$E = s(1 - m), \qquad U_0 = (1 - s) + s\, m. \tag{2.3}$$

Here s is exact (it is written directly into the exponent field), q is a relative-error minimax polynomial of degree 5 on $|r| \le \frac{1}{2} \ln 2$, and neither E nor U_0 is obtained by subtracting the other from 1. For $\Delta \to 0$, $n = 0$ and $U_0 = m$ carries full relative accuracy; for large Δ, $1 - s$ is exact. Δ is clamped at 87 inside the exponential only, where E falls below every quantity it multiplies.

In `lw_exp`, with `q(r)` the degree-5 polynomial evaluated by Horner's rule,

```
tc = min(t, 87.0)
n  = anint(tc*LOG2E)
r  = (tc - n*C1) - n*C2              ! C1 + C2 = ln 2, n*C1 exact
m  = r*q(r)                          ! 1 - exp(-r)
s  = transfer(ishft(127 - int(n), 23), 1.0)   ! 2**(-n) from its exponent bits
E  = s - s*m
u0 = (1.0 - s) + s*m
```

3 Branch-free switching

Each weight has a form that is accurate for small $|\Delta|$ (a series) and one that is accurate above some threshold (closed form in E and U_0). Both are evaluated at every point, and one is selected,

$$w = \chi\, w_{\mathrm{series}} + (1 - \chi)\, w_{\mathrm{closed}}, \qquad \chi = [\text{series range}], \tag{3.1}$$

implemented by `lw_select(a, b, first)` as a bitwise mask on the two fp32 values,

```
sel = merge(-1, 0, first)
lw_select = transfer(ior(iand(transfer(a, 1), sel),      &
                         iand(transfer(b, 1), not(sel))), 1.0)
```

so that a non-finite value in the unused form never enters the result. The closed forms are evaluated at an argument clamped into their own range where that keeps them finite.

The series are relative-error minimax polynomials, fitted on the stated interval by iteratively reweighted least squares, rounded to fp32 and then verified in compiled code against a quad-precision reference.

4 Linear scheme

With S linear in t between S_c ($t=0$) and S_u ($t=\Delta$),

$$I_c = E\,I_u + W\,S_u + \Psi\,S_c, \qquad \Psi = 1 - \frac{U_0}{\Delta}, \qquad W = U_0 - \Psi = \frac{U_0}{\Delta} - E. \tag{4.1}$$

Ψ is also the diagonal of the approximate Λ operator (divided by χ_c), and the preconditioned rates need the escape factor $1-\Psi$.

Ψ cancels as $\Delta \to 0$ and $1-\Psi$ cancels as $\Delta \to \infty$. Hence

$$\Psi = \begin{cases} \Delta\,P(\Delta), & |\Delta| \le 1, \\ 1 - U_0/\Delta, & \text{otherwise}, \end{cases} \qquad P(\Delta) = \sum_{k\ge 0} \frac{(-\Delta)^k}{(k+2)!}, \tag{4.2}$$

with P a minimax polynomial of degree 7 on $|\Delta| \le 1$. Above $|\Delta| = 1$ the escape factor is returned as U_0/Δ and W as $U_0/\Delta - E$, so that neither the $W \sim 1/\Delta$ tail nor $1-\Psi$ is formed by subtraction from 1. Every output is within 4 fp32 ulp of the exact value for $-2 \le \Delta \le 90$.

In `linear_weights`, with `lw_series(t)` $= \Delta P(\Delta)$,

```
call lw_exp(t, E, u0)
ut = u0/t
ps = lw_series(t)
thin = abs(t) <= 1.0
Psi = lw_select(ps, 1.0 - ut, thin)
Esc = lw_select(1.0 - ps, ut, thin)  ! escape factor 1 - Psi
W   = lw_select(u0 - ps, ut - E, thin)
```

and the step in the frequency loop is

```
dt = dsup05*(xxup + xx)
dt = max(dt, -real(maser_dtau_max, 4))
call linear_weights(dt, E, W, Psi, Esc)
Inu = Inuup*E + W*ssup + Psi*ss
```

The step itself is then $I_c = E\,I_u + W\,S_u + \Psi\,S_c$, followed by the maser caps on I_c and the selection of $\Psi = 0$ (no preconditioning) where $\Delta \le 0$ or $\chi_c \le 0$, all without branches.

5 Quadratic scheme

The quadratic solver is the parabolic (Bézier-equivalent) scheme of Multi3D. Measure optical depth from the upwind point, $s = \Delta - t$, and represent the source by the parabola through S_u and S_c with

slope $S'_c = \mathrm{d}S/\mathrm{d}\tau$ at the target (MC-limited from the one-sided slopes on both sides of c). With the moments

$$U_0 = \int_0^\Delta e^{-(\Delta-s)}\mathrm{d}s = 1-E, \qquad U_1 = \int_0^\Delta s\,e^{-(\Delta-s)}\mathrm{d}s = \Delta - U_0,$$
$$U_2 = \int_0^\Delta s^2 e^{-(\Delta-s)}\mathrm{d}s = \Delta^2 - 2U_1, \tag{5.1}$$

and $q = U_2/\Delta$, Eq. (1.1) becomes

$$I_c = E\,I_u + A\,S_u + B\,S_c + C\,S'_c, \qquad B = \frac{2U_1 - q}{\Delta}, \quad A = U_0 - B, \quad C = q - U_1. \tag{5.2}$$

On the last plane of a sweep the stencil is shifted one cell upwind and the slope sits at the upwind end; the same moments then give $A = q/\Delta$, $B = U_0 - A$, $C = U_1 - q$.

$U_1 \simeq \Delta^2/2$ and $q \simeq \Delta^2/3$ cancel as $\Delta \to 0$. $U_1 = \Delta\Psi$ with the Ψ of the linear scheme, so it inherits that treatment. For q,

$$q = \begin{cases} \Delta^2 R(\Delta), & |\Delta| \le 2, \\ \Delta - 2U_1/\Delta, & \text{otherwise}, \end{cases} \qquad R(\Delta) = \sum_{k\ge 0} \frac{2(-\Delta)^k}{(k+3)!}, \tag{5.3}$$

with R a minimax polynomial of degree 8 on $|\Delta| \le 2$. The series range extends to 2 rather than 1 because the closed form still loses about 10 ulp near $\Delta = -1.5$. E, U_0, U_1 and q are all within 4 ulp of the exact values for $-2 \le \Delta \le 87$ (`parab_moments`).

In `parab_moments`, with `pm_r(t)` $= R(\Delta)$,

```
call lw_exp(t, E, U0)
U1 = lw_select(t*lw_series(t), t - U0, abs(t) <= 1.0)
q  = lw_select(t*t*pm_r(t), t - 2.0*(U1/t), abs(t) <= 2.0)
```

and in the frequency loop of `long_char_quadratic`

```
rdt = 1.0/dt
call parab_moments(dt, ex, U0, U1, q)
B = (2.0*U1 - q)*rdt                  ! on ssc, the target
A = U0 - B                            ! on ssup, the upwind point
C = q - U1                            ! on dsdtau
Inu = Inuup*ex + A*ssup + B*ssc + C*dsdtau
```

The weights A, B, C are then formed in fp32. At large Δ, $A = U_0 - B \simeq 2/\Delta^2$ is a difference of two numbers close to 1 and carries an absolute error of order ε. Since $A + B = U_0$, the corresponding error in I_c is of order $\varepsilon\,S$. It is not amplified, and the Λ diagonal uses B, which has no cancellation.

6 Hermite scheme

The Hermite solver represents the source over the interval by a cubic Bézier curve in τ whose end slopes come from the chain-rule (product-rule) derivatives of χ/P_e and S/B_ν. The control points are

$$P_1 = S_u + \tfrac{\Delta}{3}S'_u, \qquad P_2 = S_c - \tfrac{\Delta}{3}S'_c, \tag{6.1}$$

each clamped into $[\min(S_u, S_c), \max(S_u, S_c)]$. The treatment of slopes, Δ (an Euler–Maclaurin trapezoid) and the clamps is unchanged; only the weights are new. With $u = t/\Delta$ and the Bernstein basis $b_0 = u^3$, $b_1 = 3u^2(1-u)$, $b_2 = 3u(1-u)^2$, $b_3 = (1-u)^3$,

$$I_c = E\,I_u + w_0 S_u + w_1 P_1 + w_2 P_2 + w_3 S_c, \qquad w_k(\Delta) = \Delta\int_0^1 b_k(u)\,e^{-\Delta u}\,\mathrm{d}u. \tag{6.2}$$

CLOSED FORM. With $M_n = \int_0^\Delta t^n e^{-t}\,\mathrm{d}t$, obtained by the recurrence $M_n = nM_{n-1} - \Delta^n E$ from $M_0 = U_0$,

$$w_0 = \frac{M_3}{\Delta^3}, \quad w_1 = \frac{3(\Delta M_2 - M_3)}{\Delta^3}, \quad w_2 = \frac{3(\Delta^2 M_1 - 2\Delta M_2 + M_3)}{\Delta^3}, \quad w_3 = M_0 - w_0 - w_1 - w_2. \tag{6.3}$$

Since $M_n \simeq \Delta^{n+1}/(n+1)$ for small Δ, these combinations lose all fp32 accuracy below $\Delta \approx 2.5$. For $\Delta < 0$, where $E > 1$ and the M_n alternate in sign, they lose it everywhere (w_2 and w_3 are off by 10^2 to 10^3 ulp at $\Delta = -2$).

SERIES AND REFLECTION. Expanding the exponential in Eq. (6.2) gives $w_k = \Delta P_k(\Delta)$ with

$$P_k(\Delta) = \sum_{n\ge 0} \frac{(-\Delta)^n}{n!} \int_0^1 b_k(u)\, u^n\,\mathrm{d}u. \tag{6.4}$$

For $\Delta > 0$ these are alternating series. b_0 and b_1 peak at the far end of the interval ($u = 1$ and $u = 2/3$), where $e^{-\Delta u}$ is smallest, so P_0 and P_1 behave like $e^{-\Delta}$ times a slowly varying factor and their series cancel strongly. The reflection $u \to 1-u$ maps b_k onto b_{3-k}:

$$\frac{w_k(\Delta)}{\Delta} = \int_0^1 b_{3-k}(v)\, e^{-\Delta(1-v)}\,\mathrm{d}v = E\,P_{3-k}(-\Delta). \tag{6.5}$$

So

$$w_0 = \Delta E\,P_3(-\Delta), \quad w_1 = \Delta E\,P_2(-\Delta), \quad w_2 = \Delta\,P_2(\Delta), \quad w_3 = \Delta\,P_3(\Delta), \tag{6.6}$$

and for $\Delta > 0$ the series for w_0 and w_1 have terms of one sign. Only two polynomials are needed: P_2 of degree 10 and P_3 of degree 9, both minimax on $|\Delta| \le 3$ (each is evaluated at $+\Delta$ and $-\Delta$). The series are used for $\Delta \le 3$ and the closed form above, where the closed form sums the weights to U_0 exactly. Every weight is within 8 ulp of the exact value for $-3 \le \Delta \le 87$ (6.5 ulp with fused multiply–add). The fp64 code it replaces used a four-term Taylor series below $\Delta = 0.05$, whose truncation error there is about 10^{-7} relative, the same order.

In `bezier3_weights`, with `b3_p2` and `b3_p3` the polynomials P_2 and P_3, the closed-form arm is evaluated at `tb = max(t, 3)`, which equals Δ wherever that arm is selected and keeps it finite elsewhere:

```
call lw_exp(t, E, M0)
tb  = max(t, 3.0)
tb2 = tb*tb
tb3 = tb2*tb
r3  = 1.0/tb3
M1 = M0 - tb*E
M2 = 2.0*M1 - tb2*E
M3 = 3.0*M2 - tb3*E
b0 = M3*r3
b1 = 3.0*(M2*tb - M3)*r3
b2 = 3.0*(M1*tb2 - 2.0*M2*tb + M3)*r3
ser = t <= 3.0
tE  = t*E
w0 = lw_select(tE*b3_p3(-t), b0, ser)
w1 = lw_select(tE*b3_p2(-t), b1, ser)
w2 = lw_select(t*b3_p2(t), b2, ser)
w3 = lw_select(t*b3_p3(t), M0 - b0 - b1 - b2, ser)
```

The interval step is then

```
call bezier3_weights(dt, ex, w0, w1, w2, w3)
Inu = Inuup*ex + w0*ssup + w1*P1 + w2*P2 + w3*ssc
```

7 Ranges and limits

- *Maser intervals.* Where $\chi < 0$, $\Delta < 0$ is capped at $-\Delta_{\max}$, set by `maser_dtau_max` (default 2). The series ranges cover this default ($|\Delta| \le 1$ and ≤ 2 for the linear and quadratic schemes, together with their closed forms, $\Delta \ge -3$ for the Bézier weights). A larger cap leaves the linear and quadratic weights accurate, but the Bézier weights lose accuracy below $\Delta = -3$.
- *Small intervals.* $|\Delta|$ is held at or above 10^{-20}, so $1/\Delta$ and Δ^2 stay finite and normal.
- *Thick intervals.* Above $\Delta = 87$, E is held at e^{-87}; the weights are otherwise unaffected.

8 Scope and verification

`long_char_linear`, `short_char`, `long_char_quadratic` and `long_char_hermite` now evaluate everything per point in fp32. `long_char_hermite2` and `short_char_hermite` use the fp32 quadratic moments but still form their weights A, B, C and, in `short_char_hermite`, the escape factor $1 - \Psi$ in fp64.

The fp32 schemes were compared with the previous fp64 weights on the solar MARCS model, for the LTE spectrum of the `sed_4000-9000` benchmark and the `mg86g` NLTE case, with and without slab decomposition of the atmosphere. Spectra differ by at most 1.6×10^{-6} relative (median $\sim 10^{-7}$), comparable to the run-to-run spread of threaded NLTE runs. On `sed_4000-9000` (one thread) the quadratic solver takes 0.30 s instead of 0.88 s and the Hermite solver 0.43 s instead of 1.40 s, because the frequency loops now vectorise in fp32.

Table 8.1: Largest error of the fp32 weights against a quad-precision reference.

Scheme	Quantities	Range of Δ	Max. error
shared	E, U_0	$-2 \ldots 87$	2.4 ulp
linear	E, W, Ψ, $1-\Psi$	$-2 \ldots 90$	4 ulp
quadratic	U_1, q	$-2 \ldots 87$	3.5 ulp
Hermite	$w_0 \ldots w_3$	$-3 \ldots 87$	7.2 ulp